Construção de Compiladores

Gramática e Parser Recursivo

Cauã Pereira Neres - 12021BCC005

2. Parte 1 — Gramática final (BNF)

Não terminal	Produção
Programa	DeclaracaoTopo Programa | ε
DeclaracaoTopo	TipoRetorno IDENT SufixoDeclaracao
TipoRetorno	Tipo | void
SufixoDeclaracao	(ParametrosOpt) Bloco | InicializacaoOpt ;
ParametrosOpt	Parametro ParametrosCauda | ε
ParametrosCauda	, Parametro ParametrosCauda | ε
Parametro	Tipo IDENT
Tipo	int | double | bool | char | string
InicializacaoOpt	= Expressao | ε
Bloco	{ ItensBloco }
ItensBloco	ItemBloco ItensBloco | ε

ItemBloco	DeclaracaoVariavel | Comando
DeclaracaoVariavel	Tipo IDENT InicializacaoOpt ;
Comando	Bloco | if (Expressao) Bloco ElseOpt | while (Expressao) Bloco | return ExpressaoOpt ; | IDENT SufixoComando
SufixoComando	= Expressao ; | (ArgumentosOpt) ;
ElseOpt	else Bloco | ε
ExpressaoOpt	Expressao | ε
ArgumentosOpt	Expressao ArgumentosCauda | ε
ArgumentosCauda	, Expressao ArgumentosCauda | ε
Expressao	Disjuncao
Disjuncao	Conjuncao DisjuncaoCauda
DisjuncaoCauda	|| Conjuncao DisjuncaoCauda | ε
Conjuncao	Igualdade ConjuncaoCauda
ConjuncaoCauda	&& Igualdade ConjuncaoCauda | ε
Igualdade	Relacional IgualdadeCauda
IgualdadeCauda	== Relacional IgualdadeCauda | != Relacional IgualdadeCauda | ε
Relacional	Aditiva RelacionalCauda
RelacionalCauda	< Aditiva RelacionalCauda | > Aditiva RelacionalCauda | <= Aditiva RelacionalCauda | >= Aditiva RelacionalCauda | ε
Aditiva	Multiplicativa AditivaCauda
AditivaCauda	+ Multiplicativa AditivaCauda | - Multiplicativa AditivaCauda | ε

Multiplicativa	Unaria MultiplicativaCauda
MultiplicativaCauda	* Unaria MultiplicativaCauda | / Unaria MultiplicativaCauda | % Unaria MultiplicativaCauda | ε
Unaria	+ Unaria | - Unaria | Primaria
Primaria	NUM | STRING | IDENT PrimariaSufixo | (Expressao) | true | false
PrimariaSufixo	(ArgumentosOpt) | ε

2. Parte 2 — Conjuntos FIRST e FOLLOW

Não terminal	FIRST	FOLLOW
Programa	{ bool, char, double, int, string, void, ε }	{ $ }
DeclaracaoTopo	{ bool, char, double, int, string, void }	{ $, bool, char, double, int, string, void }
TipoRetorno	{ bool, char, double, int, string, void }	{ IDENT }
SufixoDeclaracao	{ (, ;, = }	{ $, bool, char, double, int, string, void }
ParametrosOpt	{ ε, bool, char, double, int, string }	{) }
ParametrosCauda	{ ε, , }	{) }
Parametro	{ bool, char, double, int, string }	{), , }
Tipo	{ bool, char, double, int, string }	{ IDENT }
InicializacaoOpt	{ ε, = }	{ ; }

Bloco	{ { }	{ EOF, IDENT, bool, char, double, else, if, int, return, string, void, while, {, } }
ItensBloco	{ ε, IDENT, bool, char, double, if, int, return, string, while, { }	{ } }
ItemBloco	{ IDENT, bool, char, double, if, int, return, string, while, { }	{ IDENT, bool, char, double, if, int, return, string, while, {, } }
DeclaracaoVariavel	{ bool, char, double, int, string }	{ IDENT, bool, char, double, if, int, return, string, while, {, } }
Comando	{ IDENT, if, return, while, { }	{ IDENT, bool, char, double, if, int, return, string, while, {, } }
SufixoComando	{ (, = }	{ IDENT, bool, char, double, if, int, return, string, while, {, } }
ElseOpt	{ ε, else }	{ IDENT, bool, char, double, if, int, return, string, while, {, } }
ExpressaoOpt	{ ε, (, +, -, IDENT, NUM, STRING, false, true }	{ ; }
ArgumentosOpt	{ ε, (, +, -, IDENT, NUM, STRING, false, true }	{) }
ArgumentosCauda	{ ε, , }	{) }
Expressao	{ (, +, -, IDENT, NUM, STRING, false, true }	{), ,, ; }
Disjuncao	{ (, +, -, IDENT, NUM, STRING, false, true }	{), ,, ; }
DisjuncaoCauda	{ ε, || }	{), ,, ; }
Conjuncao	{ (, +, -, IDENT, NUM, STRING, false, true }	{), ,, ;, || }
ConjuncaoCauda	{ ε, && }	{), ,, ;, || }
Igualdade	{ (, +, -, IDENT, NUM, STRING, false, true }	{ &&,), ,, ;, || }
IgualdadeCauda	{ ε, !=, == }	{ &&,), ,, ;, || }
Relacional	{ (, +, -, IDENT, NUM, STRING, false, true }	{ !=, &&,), ,, ;, ==, || }
RelacionalCauda	{ ε, <, <=, >, >= }	{ !=, &&,), ,, ;, ==, || }

Aditiva	{ (, +, -, IDENT, NUM, STRING, false, true }	{ !=, &&,), ,, ;, <, <=, ==, >, >=, || }
AditivaCauda	{ ε, +, - }	{ !=, &&,), ,, ;, <, <=, ==, >, >=, || }
Multiplicativa	{ (, +, -, IDENT, NUM, STRING, false, true }	{ !=, &&,), +, ,, -, ;, <, <=, ==, >, >=, || }
MultiplicativaCauda	{ ε, %, *, / }	{ !=, &&,), +, ,, -, ;, <, <=, ==, >, >=, || }
Unaria	{ (, +, -, IDENT, NUM, STRING, false, true }	{ !=, %, &&,), *, +, ,, -, /, ;, <, <=, ==, >, >=, || }
Primaria	{ (, IDENT, NUM, STRING, false, true	{ !=, %, &&,), *, +, ,, -, /, ;, <, <=, ==, >, >=, || }
PrimariaSufixo	{ ε, (}	{ !=, %, &&,), *, +, ,, -, /, ;, <, <=, ==, >, >=, || }

2. Parte 3 — Verificação da condição LL(1)

Não terminal	FIRST	ε?	Condição LL(1)
Programa	{ EOF, bool, char, double, int, string, void }	não	{ bool, char, double, int, string, void } ∩ FOLLOW(Programa) = ∅
DeclaracaoTopo	{ bool, char, double, int, string, void }	não	Produção única; sem conflito entre alternativas.
TipoRetorno	{ bool, char, double, int, string, void }	não	FIRST(Tipo) e {void} são disjuntos.
SufixoDeclaracao	{ (, ;, = }	não	{(} e {;, =} são disjuntos.
ParametrosOpt	{ ε, bool, char, double, int, string }	sim	{bool, char, double, int, string} ∩ {)} = ∅
ParametrosCauda	{ ε, , }	sim	{,} ∩ {)} = ∅
Parametro	{ bool, char, double, int, string }	não	Produção única; sem conflito entre alternativas.

Tipo	{ bool, char, double, int, string }	não	Cada alternativa começa com um terminal distinto.
InicializacaoOpt	{ ε, = }	sim	{=} ∩ {;} = ∅
Bloco	{ { }	não	Produção única; sem conflito entre alternativas.
ItensBloco	{ ε, IDENT, bool, char, double, if, int, return, string, while, { }	não	FIRST(ItemBloco) ∩ FOLLOW(ItensBloco) = ∅, pois FOLLOW(ItensBloco) = {}.
ItemBloco	{ IDENT, bool, char, double, if, int, return, string, while, { }	não	FIRST de DeclaracaoVariavel e Comando são disjuntos.
DeclaracaoVariavel	{ bool, char, double, int, string }	não	Produção única; sem conflito entre alternativas.
Comando	{ IDENT, if, return, while, { }	não	As alternativas começam com terminais distintos.
SufixoComando	{ (, = }	não	{(} e {=} são disjuntos.
ElseOpt	{ ε, else }	sim	{else} ∩ FOLLOW(ElseOpt) = ∅.
ExpressaoOpt	{ ε, (, +, -, IDENT, NUM, STRING, false, true }	sim	FIRST(Expressao) ∩ {;} = ∅
ArgumentosOpt	{ ε, (, +, -, IDENT, NUM, STRING, false, true }	sim	FIRST(Expressao) ∩ {)} = ∅

ArgumentosCauda	{ ε, , }	sim	{,} ∩ {)} = ∅
Expressao	{ (, +, -, IDENT, NUM, STRING, false, true }	não	Produção única; sem conflito entre alternativas.
Disjuncao	{ (, +, -, IDENT, NUM, STRING, false, true }	não	Produção única; sem conflito entre alternativas.
DisjuncaoCauda	{ ε, || }	sim	{ || } ∩ FOLLOW(DisjuncaoCauda) = ∅
Conjuncao	{ (, +, -, IDENT, NUM, STRING, false, true }	não	Produção única; sem conflito entre alternativas.
ConjuncaoCauda	{ ε, && }	sim	{&&} ∩ FOLLOW(ConjuncaoCauda) = ∅
Igualdade	{ (, +, -, IDENT, NUM, STRING, false, true }	não	Produção única; sem conflito entre alternativas.
IgualdadeCauda	{ ε, !=, == }	sim	{!=, ==} ∩ FOLLOW(IgualdadeCauda) = ∅
Relacional	{ (, +, -, IDENT, NUM, STRING, false, true }	não	Produção única; sem conflito entre alternativas.
RelacionalCauda	{ ε, <, <=, >, >= }	sim	{<, <=, >, >=} ∩ FOLLOW(RelacionalCauda) = ∅
Aditiva	{ (, +, -, IDENT, NUM, STRING, false, true }	não	Produção única; sem conflito entre alternativas.
AditivaCauda	{ ε, +, - }	sim	{+, -} ∩ FOLLOW(AditivaCauda) = ∅
Multiplicativa	{ (, +, -, IDENT, NUM, STRING, false, true }	não	Produção única; sem conflito entre alternativas.

MultiplicativaCauda	{ ε, %, *, / }	sim	{%, *, /} ∩ FOLLOW(MultiplicativaCauda) = ∅
Unaria	{ (, +, -, IDENT, NUM, STRING, false, true }	não	{+}, {-} e FIRST(Primaria) são disjuntos.
Primaria	{ (, IDENT, NUM, STRING, false, true	não	As alternativas têm FIRST disjuntos, desde que true e false sejam terminais distintos de IDENT.
PrimariaSufixo	{ ε, (}	sim	{(} ∩ FOLLOW(PrimariaSufixo) = ∅